

Министерство образования Республики Беларусь

Учреждение образования

**БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ ИНФОРМАТИКИ
И РАДИОЭЛЕКТРОНИКИ**

Факультет компьютерных систем и сетей

Кафедра информатики

Дисциплина Операционные среды и системное программирование

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

к курсовому проекту
на тему

**РАЗРАБОТКА КОНСОЛЬНОГО НАВИГАТОРА ПО ФАЙЛОВОЙ
СИСТЕМЕ С ДРЕВОВИДНЫМ ОТОБРАЖЕНИЕМ КАТАЛОГОВ**

БГУИР КП 6-05 0612 02 018 ПЗ

Студент

Д. А. Левшуков

Руководитель

Н. Ю. Гриценко

Нормоконтролёр

Н. Ю. Гриценко

Минск 2026

СОДЕРЖАНИЕ

Введение	5
1 Архитектура программного обеспечения	6
1.1 Архитектура TUI-приложения	6
1.2 Модель представления файловой системы в памяти	7
2 Платформа программного обеспечения	9
2.1 Библиотеки управления терминалом	9
2.2 Системные API файловой системы	11
3 Теоретическое обоснование разработки программного продукта	14
3.1 Иерархическая структура файловых систем	14
3.2 Права доступа в Unix	16
4 Проектирование функциональных возможностей программы	19
4.1 Обработка ввода пользователя	19
4.2 Логика отрисовки дерева	21
4.3 Реализация файловых операций	23
5 Архитектура разрабатываемой программы	25
5.1 Описание функциональной схемы навигатора	25
5.2 Описание блок-схемы алгоритма обработки событий ввода и перерисовки экрана	26
Заключение	29
Список литературных источников	30
Приложение А (обязательное) Листинг программного кода	31
Приложение Б (обязательное) Функциональная схема навигатора	55
Приложение В (обязательное) Блок-схема алгоритма обработки событий ввода и перерисовки экрана	56
Приложение Г (обязательное) Графический интерфейс пользователя	57
Приложение Д (обязательное) Ведомость курсового проекта	58

ВВЕДЕНИЕ

На заре компьютерной эры, в эпоху операционных систем с интерфейсом командной строки, таких как CP/M и MS-DOS, управление файловой системой требовало от пользователя знания специфического синтаксиса команд и было сопряжено с высокой вероятностью ошибок.

Ситуация кардинально изменилась в 1986 году с появлением программы Norton Commander, разработанной под руководством Питера Нортон. Этот программный продукт предложил принципиально новую парадигму взаимодействия – двухпанельный интерфейс, наглядно отображающий структуру каталогов и предоставляющий доступ к основным операциям над файлами через функциональные клавиши, что стало своеобразным эталоном для целого класса программного обеспечения [1]. Удачная концепция Norton Commander породила множество клонов и альтернативных реализаций, таких как Volcov Commander, FAR Manager, Midnight Commander для Unix-подобных систем, а также DOS Navigator молдавской компании RITLabs, который расширил функциональность предшественника, добавив поддержку работы с «корзиной» и множеством панелей.

Несмотря на повсеместное распространение графических интерфейсов, интерес к консольным файловым менеджерам не угас. Их основными преимуществами остаются высокая скорость работы, минимальное потребление системных ресурсов и возможность эффективного управления файлами без использования мыши, что особенно ценится системными администраторами и разработчиками, работающими с удалёнными серверами через текстовые терминалы. Эволюция этих инструментов продолжается, они обрастают новыми функциями, сохраняя при этом свою основную идею – предоставление пользователю удобного и предсказуемого контроля над файловой системой [2]. Именно в этом контексте возникает необходимость в создании современных инструментов, сочетающих классические принципы с актуальными требованиями к надёжности и обработке ошибок.

Целью данной курсовой работы является разработка интерактивного консольного приложения-навигатора по файловой системе с древовидным отображением каталогов. Приложение должно обеспечивать базовые операции управления файлами и каталогами – просмотр, копирование, перемещение, удаление и создание – используя для этого системные вызовы, что гарантирует корректное взаимодействие с операционной системой. Ключевым требованием к разрабатываемому программному обеспечению является тщательная обработка нештатных ситуаций: программа должна явно сообщать пользователю об ошибках, связанных с недостаточными правами доступа, невозможностью доступа к занятому файлу или отсутствием запрашиваемого пути, что соответствует современным стандартам модели безопасности, управляемой дескрипторами и списками контроля доступа. Реализация проекта на языке C++ с использованием инструментов автоматизации сборки Make позволит создать эффективное решение.

1 АРХИТЕКТУРА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

1.1 Архитектура TUI-приложения

Терминальный пользовательский интерфейс (TUI) представляет собой промежуточный слой между командной строкой и графическим интерфейсом, обеспечивающий интерактивность в пределах текстового терминала. Архитектура TUI-приложения базируется на трёхуровневом разделении: управление терминалом, обработка событий и логика отображения.

Первый уровень отвечает за перевод терминала в «сырой» режим, где ввод передаётся напрямую приложению без буферизации, отключение эхо-печати и переключение на альтернативный экран для сохранения истории основного терминала. Этот уровень также реализует корректное восстановление состояния терминала при аварийном завершении.

Событийная модель современных TUI строится на асинхронной обработке ввода. Вместо синхронного опроса с блокировкой цикла отрисовки применяются каналы и отдельные потоки для чтения событий. Система различает несколько типов событий: клавиатурные комбинации с модификаторами, события мыши, сигналы изменения размера окна и пользовательские таймеры. Критически важным является внедрение тиков – фиксированных временных интервалов, определяющих частоту обновления логики приложения, что отделено от частоты перерисовки экрана. Такой подход предотвращает блокировку интерфейса при выполнении длительных операций ввода-вывода. Архитектура логики приложения в современных фреймворках следует паттерну Model-View-Update, заимствованному из Elm [3]. Модель (Model) представляет структуру данных, полностью описывающую состояние приложения в текущий момент. Функция обновления (Update) получает сообщение и текущую модель, возвращая новую модель и, опционально, команду для выполнения побочных эффектов. Команды инкапсулируют асинхронные операции – чтение файловой системы, выполнение внешних процессов, таймеры – и по завершении генерируют новое сообщение, возвращая управление в Update. Представление (View) – чистая функция, преобразующая текущую модель в строковое представление интерфейса с ANSI-последовательностями для цвета и форматирования.

Рендеринг в TUI требует оптимизации из-за ограничений терминалов. Современные реализации применяют дифференциальный рендеринг с тремя стратегиями обновления экрана. Хирургическое обновление изменяет только строки, содержимое которых действительно изменилось. Частичное обновление применяется при изменении количества строк или структурных сдвигах в видимой области – очищается область от первого изменения до конца экрана и выполняется перерисовка. Полное обновление происходит только при изменениях выше видимой области, в скролл-буфере терминала. Для измерения эффективности используются метрики: количество перерисованных строк и среднее значение на кадр. Компонентный подход реализуется через иерархическую структуру, где каждый компонент

инкапсулирует собственную логику рендеринга и обработки событий. Корневой компонент управляет дочерними элементами, делегируя фокус ввода и координируя обновления.

Для древовидного представления файловой системы используются специализированные компоненты, поддерживающие виртуальную прокрутку при работе с большими каталогами. Каждый компонент имеет уникальный идентификатор, что позволяет системе рендеринга отслеживать изменения позиций и применять оптимальную стратегию обновления. Обработка Unicode и измерение ширины символов требуют особого внимания, поскольку символы восточноазиатских языков и эмодзи занимают две позиции курсора. Современные библиотеки используют специализированные функции для корректного вычисления физической ширины строки без учёта управляющих последовательностей.

1.2 Модель представления файловой системы в памяти

Модель представления файловой системы в памяти в UNIX-подобных операционных системах строится вокруг концепции виртуальной файловой системы (VFS), которая представляет собой уровень абстракции ядра, обеспечивающий единый интерфейс для работы с различными файловыми системами. VFS скрывает от пользовательских процессов детали реализации конкретных файловых систем, предоставляя унифицированный набор системных вызовов – `open`, `read`, `write`, `stat`. Архитектура VFS реализует объектно-ориентированный подход на языке C: каждая файловая система предоставляет структуры с указателями на функции, которые реализуют операции над файловыми объектами, что позволяет ядру работать с любыми файловыми системами через единый интерфейс [4].

Центральной структурой, описывающей конкретный файл или каталог, является индексный дескриптор (`inode`). Каждый `inode` хранит полные метаданные объекта: его тип (регулярный файл, каталог, символическая ссылка, устройство), права доступа (владелец, группа, биты разрешений `rwX`), временные метки (последний доступ, модификация данных, изменение метаданных), счётчик жёстких ссылок и информацию о расположении блоков данных на диске. В Linux поля `inode` включают `i_mode`, `i_uid/i_gid`, `i_size`, `i_atime/i_ctime/i_mtime`, `i_blocks`. `Inode` существуют как на диске, так и в памяти, куда они загружаются при обращении к файлу [5].

Для ускорения преобразования имён файлов в `inode` VFS использует кэш каталогов. Объект `dentry` представляет компоненту пути – имя файла в контексте родительского каталога – и содержит указатель на соответствующий `inode`. Кэш `dentries` хранит уже разрешённые имена, что позволяет при повторном обращении по тому же пути избежать повторного чтения каталогов с диска. `Dentries` организованы в хеш-таблицы для быстрого поиска и связаны в иерархическую структуру, отражающую дерево каталогов. Поскольку `dentries` существуют только в памяти и никогда не сохраняются на диск, они обеспечивают значительный прирост производительности при

работе с файловыми путями.

Процесс разрешения пути реализуется через последовательный обход компонент имени с использованием `dcache` и `inode`-объектов. Разрешение начинается либо с корневого каталога процесса (для абсолютных путей), либо с текущего рабочего каталога (для относительных). Для каждой компоненты пути система проверяет права доступа на поиск в текущем каталоге, затем выполняет поиск соответствующей записи в `dcache`. При обнаружении символической ссылки ядро рекурсивно разрешает её содержимое. Завершающая компонента пути обрабатывается в соответствии с семантикой вызываемого системного вызова – может требоваться создание нового файла или просто получение `inode` существующего.

Трёхуровневая модель представления открытых файлов связывает процесс с конкретными файловыми объектами. На первом уровне находится процесс-специфичная таблица файловых дескрипторов, где каждому открытому файлу сопоставлен целочисленный дескриптор. Элемент этой таблицы указывает на глобальную таблицу файлов, содержащую для каждого открытого экземпляра файла текущую позицию чтения/записи, флаги открытия и указатель на `inode`. Несколько файловых дескрипторов (в том числе в разных процессах) могут ссылаться на одну запись в таблице файлов, или разные записи в таблице файлов могут ссылаться на один `inode` (при жёстких ссылках). Такая трёхуровневая организация обеспечивает разделение доступа и независимые позиции при одновременной работе с одним файлом из нескольких процессов.

Кэширование и синхронизация данных реализуются через механизмы грязных страниц и отложенной записи. `Inode`, содержимое которых было изменено в памяти, помечаются как «грязные» и помещаются в список суперблока. Периодически или при нехватке памяти ядро инициирует синхронизацию, записывая изменённые блоки на диск. Это позволяет объединять множество мелких операций записи в более крупные и эффективные транзакции. В Linux кэш `inode` организован как SLAB-кэш (`inode_cacher`), управляющий выделением и освобождением памяти под структуры `inode`, а глобальные списки `inode_in_use` и `inode_unused` поддерживают актуальное состояние всех загруженных индексных дескрипторов.

Связь между `inode` в памяти и дисковым представлением осуществляется через суперблок файловой системы, который содержит методы `alloc_inode`, `destroy_inode`, `write_inode`, `dirty_inode` и другие. Конкретная файловая система реализует эти методы, выполняя преобразование полей универсального `inode` VFS в специфичную для ФС структуру на диске. Для хранения частной информации в `inode` предусмотрено специальное поле-объединение, позволяющее каждой файловой системе прикреплять свои данные к общему VFS-`inode`. В случае файловых систем, не поддерживающих нативную концепцию `inode`, VFS-структуры строятся динамически на основе информации, извлечённой из каталогов, что полностью прозрачно для пользовательских приложений.

2 ПЛАТФОРМА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1 Библиотеки управления терминалом

Управление терминалом в UNIX-подобных системах реализуется через многоуровневую систему библиотек, где `termios` обеспечивает низкоуровневый интерфейс к драйверу терминала, а `ncurses` предоставляет высокоуровневую абстракцию для построения интерактивных приложений.

Библиотека `termios`, определенная стандартом POSIX, предоставляет единый интерфейс для управления асинхронным коммуникационным портом, независимо от конкретного аппаратного обеспечения. В основе `termios` лежит структура, которая содержит пять категорий флагов: флаги режима ввода (`c_iflag`), режима вывода (`c_oflag`), режима управления (`c_cflag`), локального режима (`c_lflag`) и массив управляющих символов `c_cc`. Флаги режима ввода управляют обработкой принимаемых символов – например, `IGNBRK` игнорирует состояние `BREAK`, `ICRNL` преобразует возврат каретки в перевод строки. Флаги режима вывода контролируют передачу данных: `OPOST` включает пост-обработку вывода, `ONLCR` отображает символ новой строки в последовательность `CR-NL`. Флаги управления (`c_cflag`) задают аппаратные параметры линии: скорость передачи (`CBAUD`), размер символа (`CS5`, `CS6`, `CS7`, `CS8`), количество стоп-битов (`CSTOPB`), контроль четности (`PARENB`) и аппаратное управление потоком (`CRTSCTS`). Локальные флаги (`c_lflag`) определяют режим обработки данных на уровне терминальной дисциплины – например, `ICANON` включает канонический режим, `ECHO` управляет отображением вводимых символов, `ISIG` разрешает генерацию сигналов при вводе специальных символов. Доступ к этим параметрам осуществляется через функции `tcgetattr()` для получения текущих атрибутов и `tcsetattr()` для их установки. Последняя позволяет выбрать момент применения изменений – немедленно, после передачи всего вывода или после сброса буферов [6].

Ключевой концепцией `termios` является понятие терминальной дисциплины – модуля ядра, обрабатывающего поток данных между аппаратным драйвером и пользовательским процессом. Стандартная `ldterm` реализует обработку в соответствии с флагами `termios` и обеспечивает два фундаментальных режима работы. В каноническом режиме, который активируется флагом `ICANON`, драйвер терминала буферизует ввод построчно, обеспечивает базовое редактирование строки и передает данные приложению только после нажатия `Enter`. Дисциплина линии также обрабатывает специальные символы: `INTR` генерирует `SIGINT`, `QUIT` – `SIGQUIT`, `SUSP` – `SIGTSTP`. Для интерактивных приложений, таких как консольный навигатор, необходим неканонический режим (`raw mode`), при котором символы передаются приложению немедленно по мере ввода, без ожидания символа перевода строки и без какой-либо предварительной обработки. Режим `cbreak` отключает каноническую обработку, но сохраняет обработку сигналов терминала, тогда как `raw mode` дополнительно отключает генерацию всех сигналов по специальным символам. Переключение в

неканонический режим требует сброса флага ICANON, отключения эховывода через сброс флага ECHO и настройки параметров VMIN и VTIME в массиве `c_cc`, которые определяют минимальное количество символов для возврата из `read()` и таймаут ожидания.

Библиотека `ncurses` построена поверх `termios` и предоставляет терминально-независимый интерфейс для чтения ввода и оптимизированного вывода на экран. При инициализации `ncurses` считывает значение переменной окружения `TERM`, определяет тип терминала и загружает соответствующее описание из базы `terminfo`.

`Terminfo` – это база данных, описывающая возможности конкретных терминалов: какие управляющие последовательности следует использовать для перемещения курсора, изменения цветов, очистки экрана, и какие функциональные клавиши поддерживаются. База содержит булевые характеристики, числовые и строковые – непосредственно управляющие последовательности с параметризацией. Благодаря этой абстракции одна и та же программа, написанная с использованием `ncurses`, корректно работает на терминалах разных типов [7].

Центральной структурой данных `ncurses` является окно, представляющее собой прямоугольную сетку символьных ячеек с координатами (y, x), где начало отсчета – левый верхний угол. Главное окно `stdscr` создается автоматически при инициализации и имеет размер текущего терминала; дополнительные окна создаются через `newwin()`. Важнейшим архитектурным решением `ncurses` является использование двойной буферизации: библиотека поддерживает две структуры данных, представляющие экран терминала – физический экран, описывающий реальное содержимое терминала, и виртуальный экран, отражающий желаемое состояние. Приложения выполняют последовательность операций записи в окна, которые модифицируют виртуальный экран, но не вызывают немедленного вывода на терминал. Когда приложение вызывает `refresh()` или `wrefresh()`, `ncurses` сначала выполняет `wnoutrefresh()`, копируя изменения из указанного окна в виртуальный экран, а затем `doupdate()`, который сравнивает виртуальный экран с физическим и вычисляет минимальный набор операций, необходимый для приведения реального экрана в желаемое состояние. Такой рендеринг минимизирует объем передаваемых данных и предотвращает мерцание – на терминал отправляются только измененные символы [8].

Для обработки ввода `ncurses` предоставляет механизм трансляции управляющих последовательностей в уникальные коды. При включенном режиме `keypad()` библиотека интерпретирует многосимвольные последовательности, генерируемые функциональными клавишами, клавишами со стрелками и клавиатурой, и возвращает через `getch()` константы. Это освобождает разработчика от необходимости вручную разбирать `escape`-последовательности, которые могут различаться на разных терминалах. Для обработки событий мыши `ncurses` предоставляет механизм активации через `mousetmask()`, позволяющий получать информацию о координатах клика, нажатых кнопках и модификаторах. Также реализована поддержка цветов

через пары «цвет переднего плана – цвет фона»: каждая пара идентифицируется номером, ассоциируется с конкретными значениями RGB, если терминал поддерживает, и активируется через COLOR_PAIR(n).

Для работы с большими объемами данных, превышающими размер экрана, ncurses предоставляет механизм pads – специальных окон, не ограниченных размером экрана. Pad создается функцией newpad(nlines, ncols) и может быть значительно больше физического экрана. Для отображения части pad на экране используется функция prefresh(), которая принимает параметры, определяющие прямоугольную область pad (pminrow, pmincol) и область экрана (sminrow, smincol, smaxrow, smaxcol), куда эта область будет выведена. Это идеально подходит для реализации прокрутки в больших каталогах, где видимая область – лишь окно в большое дерево файловой системы. В отличие от обычных окон, pads не поддерживают автоматическую прокрутку и не обновляются при операциях ввода-вывода, что требует явного вызова prefresh() после изменений.

Современные расширения ncurses включают поддержку широких символов через библиотеку ncursesw, которая оперирует типом wchar_t, позволяя работать с многобайтовыми кодировками и комбинирующими символами. Это критически важно для корректного отображения Unicode символов, которые могут занимать две позиции курсора. ncursesw учитывает ширину символов через анализ категорий Unicode и корректно обрабатывает операции вставки и удаления с такими символами. Библиотека также поддерживает работу с альтернативным экраном, что позволяет приложению временно переключаться на собственный буфер, сохраняя содержимое основного терминала для восстановления после завершения. Дополнительные библиотеки panel, form и menu, поставляемые с ncurses, реализуют управление перекрывающимися окнами и высокоуровневые элементы интерфейса.

2.2 Системные API файловой системы

Доступ к файловой системе в UNIX-подобных операционных системах реализован через набор системных вызовов, определенных стандартом POSIX. Эти вызовы предоставляют приложениям интерфейс для работы с файлами и каталогами, абстрагируясь от конкретной реализации файловой системы благодаря механизму VFS. Каждый системный вызов, такой как open(), read(), write() или stat(), транслируется через VFS в вызов соответствующей функции конкретной файловой системы, что обеспечивает единообразное поведение независимо от нижележащего носителя.

Фундаментальной абстракцией при работе с файлами является файловый дескриптор – целое неотрицательное число, которое ядро назначает каждому открытому файлу в контексте конкретного процесса. Процесс наследует три стандартных дескриптора: 0 (stdin), 1 (stdout) и 2 (stderr). Системный вызов open() создает или открывает существующий файл, принимая путь, флаги доступа и, при необходимости, права доступа, и возвращает новый файловый дескриптор. Флаги включают обязательные

режимы доступа `O_RDONLY`, `O_WRONLY`, `O_RDWR` и дополнительные флаги: `O_CREAT` для создания несуществующего файла, `O_TRUNC` для усечения, существующего до нулевой длины, `O_APPEND` для добавления в конец и `O_EXCL` для гарантии эксклюзивного создания. При указании `O_CREAT` требуется третий аргумент `mode_t`, задающий биты прав доступа через комбинацию констант `S_IRUSR`, `S_IWUSR`, `S_IRGRP` и аналогичных. Для корректной обработки ошибок приложение должно проверять возвращаемое значение: `-1` сигнализирует об ошибке с кодом в переменной `errno` [9]. Вызов `close(int fd)` освобождает файловый дескриптор. Ядро автоматически закрывает все дескрипторы при завершении процесса, однако явный вызов `close()` необходим для освобождения ресурсов в долгоживущих приложениях и корректной обработки ошибок ввода-вывода, которые могут проявиться только на момент закрытия.

Для позиционирования внутри файла используется системный вызов `lseek(int fd, off_t offset, int whence)`, который изменяет текущую позицию чтения/записи для открытого файлового дескриптора. Аргумент `whence` определяет интерпретацию смещения: `SEEK_SET` – от начала файла, `SEEK_CUR` – от текущей позиции, `SEEK_END` – от конца файла. Вызов возвращает новую позицию или `-1` при ошибке. `lseek` позволяет устанавливать позицию за текущим концом файла, создавая участки, которые не занимают место на диске и читаются как нулевые байты.

Непосредственный ввод-вывод осуществляется вызовами `read()` и `write()`. Сигнатура `read(int fd, void *buf, size_t count)` пытается считать до `count` байт в буфер и возвращает количество реально прочитанных байт, `0` при достижении конца файла или `-1` при ошибке. Аналогично `write(int fd, const void *buf, size_t count)` записывает данные из буфера и возвращает количество записанных байт, которое должно совпадать с `count` при успешной записи. Оба вызова могут вернуть меньше байт, чем запрошено, например, при чтении из `pipe` или при прерывании сигналом.

Получение метаданных файла выполняется через семейство вызовов `stat()`, которые заполняют структуру `stat` информацией о заданном файле. Для уже открытого файла используется `fstat(int fd, struct stat *buf)`, что позволяет избежать проблем с гонками при изменении пути. Структура `stat` содержит ключевые поля: `st_mode` – тип файла и права доступа, `st_ino` – номер `inode`, `st_dev` – идентификатор устройства, `st_nlink` – количество жестких ссылок, `st_uid` и `st_gid` – идентификаторы владельца, `st_size` – размер в байтах, `st_atime` – время последнего доступа, `st_mtime` – время последней модификации, `st_ctime` – время последнего изменения метаданных. Для определения типа файла предназначены макросы `S_ISREG(mode)`, `S_ISDIR(mode)`, `S_ISLNK(mode)` и другие, проверяющие соответствующие биты в `st_mode`. Права доступа проверяются маскированием `st_mode` с константами `S_IRUSR`, `S_IWUSR`, `S_IXUSR` и аналогичными.

Для навигации по каталогам предусмотрен отдельный набор вызовов. Вызов `opendir(const char *name)` открывает поток каталога и возвращает указатель на структуру `DIR`, `readdir(DIR *dirp)` последовательно читает записи

каталога, возвращая указатель на структуру `dirent`, содержащую имя файла и, в некоторых реализациях, тип файла, `closedir(DIR *dirp)` закрывает поток. Важно, что `readdir()` не гарантирует упорядоченность записей. Для перехода в другой каталог используется `chdir(const char *path)`, изменяющий текущий рабочий каталог процесса, и `getcwd(char *buf, size_t size)` для получения абсолютного пути текущего каталога.

Создание и удаление каталогов выполняется вызовами `mkdir(const char *path, mode_t mode)` и `int rmdir(const char *path)`. Удаление файлов производится через `unlink(const char *path)`, который уменьшает счетчик жестких ссылок и физически удаляет файл при достижении нуля. Переименование выполняется атомарно вызовом `rename(const char *old, const char *new)`. Для создания жестких ссылок используется `link(const char *oldpath, const char *newpath)`, символических – `symlink(const char *target, const char *linkpath)`. Чтение символической ссылки осуществляется `readlink(const char *path, char *buf, size_t bufsiz)`, возвращающим содержимое ссылки без завершающего нуля.

Управление правами доступа реализовано через `chmod(const char *path, mode_t mode)` для изменения прав и `chown(const char *path, uid_t owner, gid_t group)` для смены владельца. Проверка прав без необходимости открытия файла выполняется `access(const char *path, int amode)`, где `amode` может быть комбинацией `R_OK`, `W_OK`, `X_OK` и `F_OK` для проверки существования.

Особого внимания заслуживает трехслойная модель ядра для управления открытыми файлами, обеспечивающая корректную работу дескрипторов при вызове `fork()` и при множественных открытиях одного файла. Каждый процесс имеет собственную таблицу файловых дескрипторов, записи которой указывают на глобальную таблицу открытых файлов в ядре. Элемент таблицы файлов содержит флаги открытия и текущую позицию чтения/записи, а также указатель на `inode`. При вызове `fork()` дочерний процесс получает копию таблицы дескрипторов, указывающую на те же элементы таблицы файлов, что и родитель – таким образом, родитель и ребенок разделяют одну текущую позицию для общего файла. При независимом открытии одного и того же файла разными вызовами `open()` создаются отдельные элементы в таблице файлов с собственными позициями, хотя они указывают на один `inode`.

Обработка ошибок в системных вызовах файловой системы стандартизирована: при неудаче возвращается `-1`, а глобальная переменная `errno` устанавливается в код ошибки. Для навигатора критически важны следующие коды: `EACCES` – недостаточно прав для операции, `EBUSY` – ресурс занят, `ENOENT` – компонент пути не существует, `ENOTDIR` – компонент пути не является каталогом, `EISDIR` – попытка открыть каталог на запись, `ENOSPC` – на устройстве нет свободного места, `EROFS` – попытка записи на файловую систему, смонтированную только для чтения. Функции `error()` или `strerror()` позволяют преобразовать код ошибки в человеко-читаемое сообщение для отображения пользователю.

3 ТЕОРЕТИЧЕСКОЕ ОБОСНОВАНИЕ РАЗРАБОТКИ ПРОГРАММНОГО ПРОДУКТА

3.1 Иерархическая структура файловых систем

В операционных системах семейства Unix (включая Linux) файловая система логически организована как единое древовидное пространство имён. Корнем этого дерева является каталог, обозначаемый символом «/» (root directory). Любой объект (файл, каталог, устройство, символическая ссылка) располагается в этом дереве, и его положение однозначно задаётся путём – последовательностью имён каталогов, разделённых символом «/». Путь может быть абсолютным (начинающимся с корневого каталога) или относительным (интерпретируемым относительно текущего рабочего каталога процесса). Такая модель предоставляет пользователям и прикладным программам единообразный способ доступа к данным независимо от физического расположения файлов на носителе. По сути, это абстракция, скрывающая детали дисковых структур и реализующая принцип «всё есть файл», характерный для философии Unix [10].

Ключевым принципом построения иерархии является представление каталогов как особого типа файлов, которые содержат записи о вложенных объектах. Каждая запись каталога связывает имя файла с его индексным дескриптором (inode) – структурой, хранящей метаданные (тип, права доступа, владельца, размер, временные метки и информацию о расположении блоков данных на диске). Сами имена не хранятся в inode, а принадлежат каталогам, благодаря чему один и тот же inode может быть связан с несколькими именами (жёсткие ссылки) в разных каталогах. Каталоги, в свою очередь, формируют иерархию, поскольку каждый каталог (кроме корневого) имеет запись «...», указывающую на родительский каталог, и запись «.» – ссылку на себя. Это позволяет системе выполнять навигацию как от корня вниз, так и от текущего узла вверх. В операционной системе Linux, например, каталоги реализованы как файлы специального формата, которые могут быть прочитаны только ядром; структура записи каталога (struct dirent) определена в заголовочном файле <dirent.h> и содержит номер inode и имя файла [11].

Отличительной чертой Unix-систем является то, что физически различные носители данных (жёсткие диски, разделы, сетевые ресурсы) интегрируются в единое логическое дерево с помощью механизма монтирования. Корневая файловая система монтируется первой, а затем на любые пустые каталоги могут быть «примонтированы» другие файловые системы, в результате чего содержимое этих носителей становится доступным через соответствующие точки монтирования. Пользователь при этом не видит границ между разными устройствами: все они выглядят как непрерывная иерархия каталогов. Точка монтирования скрывает содержимое того каталога, на который она устанавливается, и вместо него отображает корень примонтированной файловой системы. В ядре Linux поддержка монтирования

реализована через структуру `vfsmount`, связывающую суперблок файловой системы с точкой монтирования в глобальном дереве. Это архитектурное решение позволяет строить единое пространство имён, обладающее практически неограниченной ёмкостью и гибкостью.

Виртуальная файловая система (VFS) представляет собой уровень абстракции ядра, который унифицирует доступ к различным конкретным файловым системам (`ext4`, `XFS`, `FAT` и др.) и предоставляет единый интерфейс системных вызовов. VFS вводит понятия суперблока – структуры, описывающей смонтированную файловую систему в целом; индексного дескриптора – структуры, описывающей конкретный объект; записи каталога (`dentry`) – структуры, кэширующей компоненты пути; и файлового объекта (`file`) – структуры, представляющей открытый экземпляр файла. При открытии файла или переходе в каталог ядро выполняет разрешение пути: последовательно обходит компоненты имени, используя кэш записей каталогов (`dentry cache`), что позволяет минимизировать обращения к диску. Для каждого компонента пути VFS вызывает операции соответствующей файловой системы (например, `ext4_lookup`), которые выполняют поиск имени в каталоге и возвращают соответствующий `inode`. Такой дизайн позволяет поддерживать множество файловых систем одновременно и обеспечивает прозрачность для пользовательских приложений.

Процесс разрешения пути в Linux начинается с определения стартового каталога: для абсолютного пути это корневой каталог процесса (обычно наследуемый от родительского процесса), для относительного – текущий рабочий каталог. Каждый процесс хранит эти ссылки в своей структуре `task_struct`: указатель `fs_struct` содержит ссылки на корень и текущий рабочий каталог. По мере разбора пути VFS использует механизм кэширования `dentry`, который хранит имена и соответствующие им `inode` в хеш-таблицах. Если требуемая запись отсутствует в кэше, ядро обращается к конкретной файловой системе для её загрузки с диска. При этом учитываются символические ссылки: если встретившаяся компонента является символической ссылкой, ядро рекурсивно разрешает её содержимое до тех пор, пока не будет достигнут реальный объект (с ограничением на глубину вложенности для предотвращения бесконечных циклов).

Каждый процесс имеет текущий рабочий каталог (`current working directory`), относительно которого интерпретируются относительные пути. Этот каталог хранится в структуре `fs_struct` процесса и может быть изменён системным вызовом `chdir()` или `fchdir()`. При вызове `getcwd()` ядро возвращает абсолютный путь от корня, рекурсивно поднимаясь по родительским ссылкам `dentry`, пока не достигнет корневого каталога процесса. В разработанном файловом навигаторе работа с путями построена на использовании абсолютных путей, получаемых через `realpath()` для нормализации символических ссылок и относительных ссылок, а также через `getcwd()` для получения текущего каталога. Это гарантирует корректную навигацию даже при наличии сложной структуры монтирования и символических ссылок, полностью соответствуя поведению стандартных утилит Unix.

В современных Unix-подобных системах модель иерархического пространства имён получила дальнейшее развитие. В Linux, например, существуют пространства имён монтирования (mount namespaces), позволяющие разным процессам иметь различное представление о дереве файловой системы, что лежит в основе контейнерной виртуализации. Механизм bind mounts позволяет примонтировать один и тот же каталог в нескольких точках, создавая альтернативные пути доступа к данным. Несмотря на эти усложнения, базовое представление о едином древовидном пространстве имён, восходящее к классическим Unix-системам, остаётся неизменным и является фундаментом, на котором строятся все операции управления файлами, включая те, что реализованы в консольном навигаторе.

3.2 Права доступа в Unix

В операционных системах семейства Unix управление доступом к объектам файловой системы базируется на дискреционной модели, в которой каждый объект (файл, каталог, устройство) связывается с набором прав, определяющих, какие операции разрешены для различных категорий пользователей. Эта модель реализована на уровне индексных дескрипторов (inode) и является неотъемлемой частью архитектуры безопасности, обеспечивая изоляцию данных между пользователями и процессами. Исторически сложившаяся схема прав доступа (rwx) была введена в ранних версиях Unix и сохранилась в современных системах благодаря своей простоте и эффективности.

Традиционная схема прав доступа в Unix оперирует тремя категориями субъектов: владелец (owner) – пользователь, создавший объект или назначенный владельцем через системный вызов chown; группа (group) – группа, которой принадлежит объект; остальные (others) – все остальные пользователи системы. Для каждой категории определены три бита разрешений: чтение (r – read), запись (w – write) и выполнение (x – execute). Для регулярных файлов бит чтения позволяет открыть файл и прочитать его содержимое, бит записи – модифицировать или удалить файл (при наличии прав на запись в каталог), бит выполнения – запустить файл как программу или скрипт. Для каталогов семантика битов отличается: чтение позволяет получить список содержимого каталога (например, через readdir), запись – создавать и удалять файлы внутри каталога, выполнение (часто называемое «поиск») – проходить через каталог (то есть открывать вложенные объекты по имени). Отсутствие бита выполнения на каталоге делает его содержимое недоступным даже при наличии бита чтения, поскольку ядро не может разрешить пути внутри такого каталога.

Права доступа хранятся в поле st_mode структуры stat в виде комбинации битовых флагов, определённых в заголовочном файле <sys/stat.h>. Для удобства работы с ними предусмотрены макросы: S_ISREG(mode), S_ISDIR(mode) и другие для определения типа объекта, а также маски для выделения прав: S_IRUSR, S_IWUSR, S_IXUSR (для

владельца), `S_IRGRP`, `S_IWGRP`, `S_IXGRP` (для группы), `S_IROTH`, `S_IWOTH`, `S_IXOTH` (для остальных). Эти флаги могут быть установлены или сброшены с помощью системных вызовов `chmod()` и `fchmod()`. При выводе списка файлов утилитой `ls` права отображаются в виде строки из десяти символов: первый символ определяет тип объекта (например, «-» для файла, «d» для каталога), следующие три символа – права владельца, затем три – права группы, последние три – права остальных.

Помимо основных битов прав, в Unix существуют три специальных бита, изменяющих поведение при выполнении. Бит `setuid` (`S_ISUID`), установленный на исполняемом файле, заставляет процесс, запускающий этот файл, временно получить эффективный идентификатор пользователя, равный владельцу файла, вместо реального идентификатора запустившего пользователя. Аналогично, бит `setgid` (`S_ISGID`) для файлов меняет эффективную группу; для каталогов бит `setgid` означает, что новые файлы, создаваемые внутри такого каталога, наследуют группу каталога, а не группу создающего процесса. Бит `sticky` (`S_ISVTX`), исторически использовавшийся для удержания сегментов программ в свопе, в современных системах применяется для каталогов, доступных на запись многим пользователям (например, `/tmp`): если бит `sticky` установлен, удалить или переименовать файл внутри такого каталога может только владелец файла, владелец каталога или суперпользователь.

При создании нового файла или каталога начальные права доступа определяются маской создания файлов (`umask`) – значением, хранящимся в контексте процесса. `Umask` представляет собой набор битов, которые будут сброшены в правах, запрошенных при создании. Типичная `umask 022` означает, что биты записи для группы и остальных будут отключены, что даёт права 755 для каталогов и 644 для файлов (если создающий процесс не указал иные права). Системные вызовы `open()` с флагом `O_CREAT` и `mkdir()` принимают аргумент `mode`, который, после применения `umask`, определяет итоговые права объекта.

Проверка прав доступа в Unix выполняется ядром при каждой операции, требующей доступа к объекту файловой системы. Алгоритм проверки для процесса выглядит следующим образом: если эффективный идентификатор пользователя процесса равен 0 (суперпользователь), доступ разрешён практически всегда (исключение – файлы, для которых даже суперпользователь не может выполнить определённые действия, например, запись в файл, смонтированный только для чтения). В противном случае ядро проверяет, совпадает ли эффективный идентификатор пользователя с идентификатором владельца объекта; если да, применяются биты прав владельца. Если нет, проверяется совпадение эффективной группы процесса с группой объекта (или принадлежность процесса к этой группе); если да, применяются биты прав группы. Во всех остальных случаях применяются биты прав для остальных. Для выполнения операции требуются все необходимые биты: например, для чтения файла нужен бит чтения на сам файл и биты выполнения на все каталоги в пути к нему.

Важным аспектом является различие между реальными и эффективными идентификаторами пользователя и группы. Реальные идентификаторы (ruid, rgid) определяют фактического владельца процесса, в то время как эффективные (euid, egid) используются для проверки прав доступа. При запуске обычной программы эффективные идентификаторы совпадают с реальными; однако при установке битов `setuid/setgid` эффективные идентификаторы временно заменяются на идентификаторы владельца файла. Это позволяет выполнять привилегированные операции с контролируемым повышением прав.

В современных Unix-подобных системах базовая модель `rwX` дополнена более гибкими механизмами. Списки контроля доступа (ACL, Access Control Lists), реализованные в стандарте POSIX.1e (в настоящее время черновик), позволяют задавать детализированные правила для произвольных пользователей и групп, а также определять права по умолчанию для вновь создаваемых объектов внутри каталога. Кроме того, в Linux внедрена система `capabilities`, которая разбивает привилегии суперпользователя на отдельные права (например, `CAP_DAC_OVERRIDE` для обхода проверок прав доступа), что позволяет предоставлять процессам ограниченный набор привилегий без полного `root`-доступа [10]. Несмотря на наличие этих расширений, базовая модель `rwX` остаётся фундаментом, на котором строятся все более сложные механизмы, и именно она используется в большинстве консольных утилит, включая разработанный файловый навигатор, поскольку системные вызовы `stat`, `chmod`, `open`, `mkdir`, `unlink`, `rename` оперируют именно этой моделью, обеспечивая переносимость и совместимость с широким спектром файловых систем.

При копировании файла (операция, реализованная в навигаторе) необходимо учитывать права доступа к исходному файлу (чтение) и к целевому каталогу (запись и выполнение). В программе используется системный вызов `open()` с `O_RDONLY` для исходного файла и `open()` с `O_WRONLY|O_CREAT|O_TRUNC` для целевого, при этом права для создаваемого файла задаются явно (0644), что соответствует традиционной практике создания файлов с правами `rw-r--r--`. При копировании каталога программа рекурсивно обходит исходное дерево, создавая целевые каталоги с правами 0755 (`rwxr-xr-x`). Если на каком-либо этапе недостаточно прав (например, нет доступа на чтение исходного файла или на запись в целевой каталог), системный вызов возвращает -1, а `errno` устанавливается в `EACCESS`. Навигатор перехватывает эту ситуацию, выводит сообщение об ошибке в статусную строку и записывает детали в лог, что соответствует требованиям надёжной обработки ошибок. Удаление файла или каталога, в свою очередь, требует прав на запись и выполнение в родительском каталоге (чтобы изменить его содержимое) и, в случае удаления файла, обычно не требует прав на сам файл, однако программа дополнительно запрашивает подтверждение, чтобы избежать случайного удаления защищённых объектов.

4 ПРОЕКТИРОВАНИЕ ФУНКЦИОНАЛЬНЫХ ВОЗМОЖНОСТЕЙ ПРОГРАММЫ

В качестве языка реализации выбран C++ стандарта C++17, что позволило использовать современные возможности языка (умные указатели, лямбда-выражения, контейнеры стандартной библиотеки) при сохранении полного контроля над системными вызовами и управлением памятью. Компиляция осуществляется с помощью GCC (g++), поддерживающего все необходимые стандарты и обеспечивающего высокую производительность результирующего кода. Сборка проекта автоматизирована посредством Makefile, который определяет зависимости между исходными файлами и позволяет выполнять сборку одной командой `make`, что упрощает процесс компиляции и воспроизведения результатов.

Для построения текстового пользовательского интерфейса (TUI) использована библиотека `ncursesw`. Эта библиотека предоставляет терминально-независимый интерфейс для управления экраном, обработки клавиатурного ввода (включая функциональные клавиши и стрелки) и оптимизированного вывода с двойной буферизацией. Использование `ncursesw` критически важно для корректной работы с UTF-8, поскольку стандартные функции работы с однобайтовыми символами не способны правильно обрабатывать многобайтовые кодировки и символы, занимающие две позиции курсора. В программе также активно применяются системные вызовы POSIX для работы с файловой системой: `open`, `read`, `write`, `stat`, `opendir`, `readdir`, `mkdir`, `rmdir`, `unlink`, `rename`, `getcwd`, `chdir`, `realpath` и другие. Эти вызовы обеспечивают прямое взаимодействие с ядром операционной системы без дополнительных уровней абстракции, что гарантирует прозрачность операций и точную обработку ошибок. Для преобразования между многобайтовой кодировкой UTF-8 и внутренним представлением в виде широких строк (`wstring`) используются стандартные функции `mbstowcs` и `wcstombs`, что обеспечивает корректное отображение имён файлов на любом языке.

Структура проекта включает один основной исходный файл (`main.cpp`), содержащий полную реализацию навигатора, и Makefile для автоматизации сборки. В коде применён объектно-ориентированный подход: определены класс `FileManager`, инкапсулирующий все аспекты работы интерфейса и файловых операций, а также вспомогательные структуры `TreeNode` и `FileEntry`, представляющие элементы файловой системы в режиме дерева и плоского списка соответственно. Логирование всех операций и ошибок осуществляется в файл `file_manager.log` с использованием стандартного потока вывода `std::ofstream`, что позволяет в дальнейшем анализировать работу программы и диагностировать возможные сбои.

4.1 Обработка ввода пользователя

Обработка ввода в разработанном навигаторе реализована с

использованием библиотеки `ncursesw`, которая предоставляет высокоуровневый интерфейс для чтения клавиатурных событий в неканоническом режиме. При инициализации приложения выполняются вызовы `initscr()` для инициализации терминала, `cbreak()` для отключения буферизации строк (каждый введённый символ передаётся приложению немедленно), `noccho()` для подавления автоматического вывода нажатых клавиш и `keypad(stdscr, TRUE)` для включения трансляции управляющих последовательностей функциональных клавиш и клавиш со стрелками в специальные коды (константы `KEY_UP`, `KEY_DOWN`, `KEY_ENTER` и т.д.). Вызов `curs_set(0)` скрывает курсор, что соответствует стандартному поведению консольных файловых менеджеров. Для поддержки широких символов используется вариант библиотеки `ncursesw`, а также предварительно установлена локаль вызовом `setlocale(LC_ALL, "")`, что позволяет корректно обрабатывать многобайтовые последовательности UTF-8.

Основной цикл обработки событий реализован в методе `FileManager::run()`. После инициализации и отрисовки начального интерфейса программа входит в бесконечный цикл, на каждой итерации вызывая `getch()` для блокирующего чтения одного символа или управляющей последовательности. Полученное значение сравнивается с константами, определёнными в `ncurses`, а также с ASCII-кодами букв. Для обеспечения нечувствительности к регистру команды (`c`, `m`, `d`, `n`, `t`, `q`) введённый символ приводится к нижнему регистру с помощью `tolower()`. Коды управляющих клавиш (`KEY_UP`, `KEY_DOWN`, `KEY_ENTER`, `KEY_BACKSPACE`, `KEY_RESIZE`) обрабатываются непосредственно в операторе `switch`. При получении символа `'q'` цикл прерывается и выполняется корректное завершение работы с вызовом `endwin()`.

Навигация по списку элементов реализована с помощью клавиш `KEY_UP` и `KEY_DOWN`. При нажатии вверх переменная `selectedIndex` уменьшается на единицу (если не достигнуто начало списка), при нажатии вниз – увеличивается на единицу (если не достигнут конец). После изменения индекса вызывается функция `adjustScroll()`, которая корректирует значение `scrollOffset` так, чтобы выбранный элемент оставался в пределах видимой области окна. Это обеспечивает плавную прокрутку списка при навигации.

Клавиша `KEY_ENTER` (или символ `\n`) обрабатывается в зависимости от текущего режима отображения. В режиме дерева (`treeMode == true`) нажатие `Enter` на каталоге приводит к вызову методов `loadChildren()` (если узел свёрнут) или `unloadChildren()` (если раскрыт), после чего обновляется плоский список `flatList` и сохраняется текущая позиция выбора. Если выбран файл, в статусной строке отображается его имя, но никакой операции не выполняется. В плоском режиме нажатие `Enter` на каталоге приводит к смене текущего каталога: формируется новый путь `currentPath`, вызывается `loadFlatDirectory()` для перезагрузки содержимого, и индекс выбора сбрасывается в ноль. Если выбран файл, также выводится сообщение в статусную строку.

Клавиша `KEY_BACKSPACE` реализует переход на уровень выше. В режиме дерева происходит смена текущего каталога на родительский (путем

обрезания последней компоненты пути), после чего дерево полностью перестраивается вызовом `loadTree()`. В плоском режиме аналогично изменяется `currentPath` и перезагружается список файлов. При достижении корневого каталога выводится предупреждение в статусную строку.

Для реализации команд `c` (копирование), `m` (перемещение), `d` (удаление) и `n` (создание) программа использует вспомогательный метод `inputStringWithDefault()`, который создаёт модальное окно ввода с редактированием строки. Этот метод активирует эхо-вывод (`echo()`), показывает курсор (`curs_set(1)`), создаёт новое окно `newwin()` с рамкой, выводит приглашение и текущее значение по умолчанию, а затем в цикле обрабатывает символы, позволяя пользователю редактировать строку с поддержкой клавиш стрелок влево/вправо, `Backspace`, `Delete` и `Ctrl+U` (очистка всей строки). После нажатия `Enter` окно удаляется (`delwin`), эхо и курсор отключаются, и введённое значение возвращается вызывающему коду. Такой подход обеспечивает привычный интерфейс для ввода путей и имён файлов без необходимости переключения в отдельный режим.

Особое внимание уделено обработке события изменения размера терминала. При получении кода `KEY_RESIZE` вызывается метод `refreshAfterResize()`, который переопределяет размеры окон `listWin` и `statusWin` с учётом новых значений `maxY` и `maxX`, полученных через `getmaxyx()`, и выполняет повторную отрисовку. Это позволяет интерфейсу адаптироваться к изменению размера окна без потери данных и без перезапуска программы. Вся обработка ввода выполнена в одном потоке без асинхронных операций, что упрощает логику и исключает состояния гонки, однако при длительных файловых операциях (например, копировании большого каталога) интерфейс временно блокируется, что является ограничением текущей реализации и может быть устранено в будущем путём внедрения фоновых потоков.

4.2 Логика отрисовки дерева

Отрисовка иерархического дерева каталогов в разработанном навигаторе основана на модели, где каждый узел (каталог или файл) представлен структурой `TreeNode`. Узел содержит имя (`std::wstring`), полный путь (`std::string`), флаг `isDirectory`, флаг `expanded` (раскрыт ли каталог), вектор умных указателей на дочерние узлы (`std::vector<std::unique_ptr<TreeNode>>` `children`) и указатель на родительский узел. Такая структура позволяет рекурсивно обходить дерево и динамически загружать содержимое каталогов только при необходимости (ленивая загрузка) – метод `loadChildren()` открывает каталог через `opendir()`, читает записи `readdir()`, для каждой записи вызывает `stat()` для получения метаданных, создаёт дочерний узел и добавляет его в вектор. После загрузки узлы сортируются: каталоги идут первыми, затем файлы; внутри каждой группы – по алфавиту. Метод `unloadChildren()` очищает вектор и сбрасывает флаг `expanded`, что позволяет освободить память при сворачивании узла.

Для отображения дерева в ограниченной области терминала программа

формирует плоский список `flatList`, который представляет собой линейное представление дерева с учётом состояния раскрытия узлов. Построение выполняется рекурсивной функцией `buildFlatListFromTree()`: она добавляет текущий узел в список, затем, если узел является каталогом и раскрыт, обходит всех его детей в порядке, определённом при загрузке. После изменений (раскрытие/сворачивание узла, переключение режима, операции, изменяющие файловую систему) вызывается `updateFlatList()`, которая перестраивает список.

Вывод дерева на экран осуществляется в методе `displayList()`. Для каждого элемента из `flatList` вычисляется глубина вложенности `depth` путём подъёма по указателям `parent` до корня. На основе глубины формируется строка отступа из пробелов (два пробела на уровень). Для каталогов в зависимости от состояния `expanded` добавляется префикс `[-]` (раскрыт) или `[+]` (свёрнут), для файлов – отступ из четырёх пробелов. Полная строка выравнивается по ширине окна с помощью функции `truncateWithEllipsis()`, которая укорачивает строку до максимальной длины и добавляет три точки, если длина превышает допустимую. При этом учитывается, что если максимальная длина меньше 4, обрезка выполняется без добавления многоточия. Для работы с широкими символами используются функции `mvwaddwstr()` и соответствующие типы `wchar_t`.

Визуальное выделение текущего выбранного элемента выполняется через атрибут `A_REVERSE`. При проходе по элементам `flatList` (от `scrollOffset` до `scrollOffset + visibleRows`) проверяется, совпадает ли индекс с `selectedIndex`. Если да, включается атрибут реверса, выводится строка, затем атрибут отключается. В противном случае строка выводится обычным образом. Такой подход минимизирует количество вызовов `wattron/wattroff`.

Управление прокруткой обеспечивается функцией `adjustScroll()`, вызываемой после каждого изменения `selectedIndex`. Она вычисляет количество видимых строк `visibleRows` как `maxY - 3` (высота окна списка, так как три строки отведены под статусную строку). Если выбранный индекс меньше `scrollOffset`, сдвиг устанавливается на выбранный индекс; если индекс больше или равен `scrollOffset + visibleRows`, сдвиг устанавливается на `selectedIndex - visibleRows + 1`. После коррекции проверяется, чтобы `scrollOffset` не выходил за границы списка. Такой механизм обеспечивает, что выбранный элемент всегда остаётся в видимой области.

При нажатии `Enter` в режиме дерева происходит переключение состояния узла: если выбранный узел – каталог, то в зависимости от текущего флага `expanded` вызывается `loadChildren()` (загрузка дочерних элементов) или `unloadChildren()` (освобождение памяти). После изменения состояния вызывается `updateFlatList()` для перестроения плоского списка. Для сохранения позиции выбора программа после перестроения находит новый индекс узла в обновлённом `flatList` (поскольку список мог изменить длину и смещение). Если узел был раскрыт, его дети появляются в списке, и позиция выбора остаётся на том же узле; если свёрнут, дети исчезают, и выбор остаётся на сворачиваемом узле. Такой подход обеспечивает интуитивно понятное

поведение при навигации по дереву.

Переключение между режимами отображения (дерево / плоский список) происходит по нажатию клавиши `t`. При переключении сбрасывается индекс выбора и смещение прокрутки; в зависимости от нового режима вызывается либо `updateFlatList()`, либо `loadFlatDirectory()`. При этом статусная строка обновляется, отображая текущий режим.

Все операции отрисовки выполняются с использованием двойной буферизации `ncurses`: сначала изменения вносятся в виртуальный экран через `mvwaddwstr()` и `wrefresh()`, который затем сравнивает виртуальный экран с физическим и отправляет на терминал только изменённые символы. Это минимизирует мерцание и ускоряет перерисовку, что особенно важно при работе с большими списками файлов.

4.3 Реализация файловых операций

Реализация файловых операций выполнена с использованием системных вызовов `POSIX`, что обеспечивает прямое взаимодействие с ядром операционной системы и гарантирует корректную обработку ошибок на низком уровне. Все операции реализованы в виде методов класса `FileManager` и используют единый подход: перед выполнением проверяется существование исходного объекта, затем вызывается соответствующий системный вызов или рекурсивная функция, после чего в случае успеха обновляется отображаемая, а в случае ошибки формируется сообщение, отображаемое в статусной строке, и выполняется запись в лог-файл.

Копирование реализовано отдельно для файлов и для каталогов. Функция `copyFile()` принимает пути источника и назначения. Исходный файл открывается системным вызовом `open()` с флагом `O_RDONLY`. Целевой файл создаётся с флагами `O_WRONLY | O_CREAT | O_TRUNC` и правами доступа `0644 (rw-r--r--)`. Копирование данных выполняется в цикле: чтение блоками через `read()` и запись через `write()`. Если все байты успешно записаны, оба дескриптора закрываются, и операция считается успешной. В случае любой ошибки возвращается `false`, а в статусную строку выводится сообщение, сформированное из кода ошибки `errno` с помощью `strerror()`. Копирование каталога реализовано рекурсивной функцией `copyDirectory()`. Сначала вызывается `mkdir()` для создания целевого каталога с правами `0755`. Затем исходный каталог открывается через `opendir()`, и для каждой записи (кроме `.` и `..`) рекурсивно вызывается либо `copyDirectory()`, если это подкаталог, либо `copyFile()`, если это файл. Рекурсия продолжается до полного копирования всей иерархии. В случае ошибки на любом уровне операция прерывается, возвращается `false`, и ошибка логируется. При успешном копировании каталога запись в лог содержит информацию об исходном и целевом пути.

Для перемещения и переименования используется системный вызов `rename()`. Функция `rename()` атомарно заменяет имя файла или перемещает его в другой каталог. В разработанном навигаторе перед вызовом `rename()` пользователю предлагается ввести новый путь или имя с помощью функции

`inputStringWithDefault()`, при этом по умолчанию подставляется текущее имя выбранного объекта. После выполнения проверяется возвращаемое значение: при успехе обновляется отображение, при ошибке в статусную строку выводится сообщение с описанием ошибки и выполняется запись в лог. Пользователь может указать как полный путь, так и просто новое имя в текущем каталоге; `rename()` корректно обрабатывает оба варианта.

Удаление файлов выполняется системным вызовом `unlink()`, который уменьшает счётчик жёстких ссылок и удаляет файл, когда счётчик достигает нуля. Для удаления каталогов реализована рекурсивная функция `removeDirectory()`. Сначала каталог открывается через `opendir()`, после чего для каждой записи (кроме `.` и `..`) определяется её тип с помощью `stat()`. Если запись является подкаталогом, вызывается `removeDirectory()` рекурсивно; если файлом – вызывается `unlink()`. После очистки содержимого вызывается `rmdir()` для удаления самого каталога. Такая рекурсивная стратегия гарантирует, что каталог будет удалён только после удаления всех вложенных объектов. Перед выполнением удаления программа запрашивает подтверждение через `inputStringWithDefault()`. Если пользователь вводит `y` или `Y`, операция выполняется; в противном случае она отменяется.

Создание нового файла выполняется вызовом `open()` с флагами `O_WRONLY | O_CREAT | O_EXCL`. Флаг `O_EXCL` гарантирует, что файл не будет перезаписан, если уже существует; в этом случае `open()` возвращает ошибку с кодом `EEXIST`. Права доступа для создаваемого файла устанавливаются в `0644`. После успешного создания файл сразу закрывается. Для создания каталога используется вызов `mkdir()` с правами `0755`. Путь для создания определяется функцией `getCreationPath()`, которая возвращает каталог, в котором должен быть создан объект. Пользователь вводит тип создаваемого объекта (`f` или `d`) и его имя через `inputStringWithDefault()`.

Все операции (успешные и неудачные) записываются в файл `file_manager.log`, который открывается в режиме добавления (`std::ios::app`) при запуске программы. Для записи используется функция `log()`, которая принимает строку в формате UTF-8 и выводит её в лог-файл с переводом строки. Логи содержат информацию о выполненных действиях (копирование, перемещение, удаление, создание) с указанием путей, а также тексты ошибок, полученные из `strerror(errno)`. Это позволяет в дальнейшем анализировать работу программы и диагностировать проблемы. При завершении программы лог-файл корректно закрывается.

Каждый системный вызов сопровождается проверкой возвращаемого значения. В случае ошибки (возврат `-1` или `NULL`) программа не аварийно завершается, а формирует сообщение, которое выводится в статусную строку (в виде широкой строки `std::wstring`) и записывается в лог. Использование `errno` и `strerror()` обеспечивает получение понятного текста ошибки (например, "Permission denied", "No such file or directory"), который информирует пользователя о причине сбоя. Такой подход соответствует современным стандартам надёжности и обеспечивает предсказуемое поведение программы в нештатных ситуациях.

5 АРХИТЕКТУРА РАЗРАБАТЫВАЕМОЙ ПРОГРАММЫ

5.1 Описание функциональной схемы навигатора

Функциональная схема файлового навигатора отражает общую структуру работы системы в виде взаимосвязанных функциональных блоков, реализующих основные этапы обработки данных.

Работа системы начинается с блока запуска программы, в котором осуществляется передача стартового пути и переход к этапу инициализации.

После запуска выполняется инициализация системы, включающая настройку окружения ncurses, локализацию, запуск логирования и создание окон пользовательского интерфейса.

Далее выполняется загрузка файловой структуры, в ходе которой происходит чтение содержимого указанного каталога и формирование внутреннего представления данных. В зависимости от выбранного режима работы формируется либо древовидная структура каталогов, либо плоский список файлов.

После формирования структуры данных осуществляется отображение интерфейса, включающее вывод списка файлов и каталогов, а также статусной строки, содержащей информацию о текущем состоянии системы и доступных командах пользователя. При этом обеспечивается согласованность отображаемых данных с текущим состоянием файловой системы, включая выбранный режим навигации и активный элемент списка.

Основной режим работы системы реализован через блок обработки пользовательского ввода, в рамках которого происходит считывание нажатых клавиш и определение типа выполняемого действия. В зависимости от введённой команды управление передаётся в соответствующие функциональные блоки.

При навигационных действиях выполняется перемещение по структуре файловой системы: переход по каталогам, возврат в родительские директории, а также переключение режима отображения. При выборе файловых операций выполняются действия по созданию, копированию, перемещению и удалению файлов и каталогов.

После выполнения любой операции осуществляется обновление внутреннего состояния системы, включая пересчёт структуры файлов, обновление списка отображаемых элементов и восстановление состояния раскрытых каталогов в древовидном режиме. Далее выполняется перерисовка интерфейса, обеспечивающая актуальное отображение данных и синхронизацию визуального представления с текущими изменениями файловой системы.

Цикл обработки пользовательского ввода повторяется до момента получения команды завершения работы. При выходе из программы выполняется корректное завершение всех подсистем, включая освобождение ресурсов, закрытие интерфейса ncurses и завершение работы системы логирования.

Таким образом, функциональная схема отражает модульную организацию файлового навигатора и демонстрирует разделение системы на логические блоки, обеспечивающие обработку данных, взаимодействие с пользователем и выполнение файловых операций. Система функционирует в событийно-ориентированном режиме, где каждое действие пользователя инициирует соответствующее изменение состояния и последующее обновление отображения.

Функциональная схема представлена в Приложении Б.

5.2 Описание блок-схемы алгоритма обработки событий ввода и перерисовки экрана

Блок-схема алгоритма обработки событий ввода и перерисовки экрана отражает общую структуру работы файлового менеджера в интерактивном режиме. Основу алгоритма составляет бесконечный цикл обработки пользовательского ввода, в котором осуществляется чтение нажатой клавиши, анализ события, выполнение соответствующей операции и последующее обновление интерфейса.

После инициализации системы (настройка ncurses, локализации, логирования и загрузка начальных данных файловой системы) программа переходит в основной цикл ожидания ввода пользователя. В этом цикле происходит последовательное считывание символа или управляющей клавиши.

В зависимости от типа введённого события выполняется одна из групп действий: навигация по списку файлов и каталогов, открытие или сворачивание директорий, переход к родительскому каталогу, переключение режима отображения (дерево/плоский список), а также файловые операции (создание, копирование, перемещение и удаление объектов файловой системы).

После выполнения любой операции выполняется обновление внутреннего состояния программы. При этом пересчитываются структуры данных, отвечающие за отображение файлов (включая формирование списка элементов и восстановление состояния раскрытых каталогов в режиме дерева), а также корректируется позиция прокрутки списка для обеспечения корректного отображения выбранного элемента. Далее выполняется перерисовка интерфейса, включающая обновление области списка файлов и статусной строки.

Цикл обработки событий повторяется до момента нажатия пользователем клавиши выхода, после чего выполняется корректное завершение работы программы и освобождение ресурсов (завершение работы ncurses, закрытие файловых дескрипторов и лог-файла).

Таким образом, алгоритм реализует событийно-ориентированную модель взаимодействия с пользователем, обеспечивая непрерывное обновление интерфейса в ответ на действия пользователя.

Блок-схема алгоритма представлена в Приложении В.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта разработан интерактивный консольный навигатор по файловой системе с древовидным отображением каталогов. Программа реализована на языке C++ с использованием стандарта C++17, библиотеки `ncursesw` для построения терминального пользовательского интерфейса и системных вызовов POSIX для выполнения файловых операций. Сборка проекта автоматизирована посредством `Makefile`, что обеспечивает простоту компиляции и воспроизводимость результатов.

Разработанное приложение предоставляет пользователю два режима отображения: иерархическое дерево каталогов с возможностью раскрытия и сворачивания узлов, а также плоский список текущего каталога. Навигация по списку осуществляется с помощью клавиш управления курсором; поддерживается переход в родительский каталог, раскрытие и сворачивание подкаталогов, а также переключение между режимами отображения. Реализованы основные операции управления файлами: копирование (с поддержкой рекурсивного копирования каталогов), перемещение (переименование), удаление (рекурсивное удаление каталогов) и создание файлов и каталогов. Все операции выполняются с использованием системных вызовов `open`, `read`, `write`, `stat`, `opendir`, `readdir`, `mkdir`, `rmdir`, `unlink`, `rename`, что гарантирует прямое взаимодействие с ядром операционной системы.

Особое внимание уделено обработке нештатных ситуаций: при недостатке прав доступа, отсутствии файла или занятости ресурса программа не завершается аварийно, а выводит понятное сообщение об ошибке в статусную строку и записывает детали в лог-файл. Поддержка UTF-8 обеспечивается использованием широких символов и функций преобразования `mbstowcs/wcstombs`, что позволяет корректно отображать имена файлов на любых языках.

Разработанный навигатор успешно протестирован на операционной системе Linux (Ubuntu) в среде терминала. Проведены тесты на корректность завершения (код возврата 0), производительность (копирование больших файлов). Во всех случаях программа демонстрировала стабильную работу и предсказуемое поведение.

Документация к проекту представлена в двух форматах: файл `README.md` содержит описание функциональности, требования к системе, инструкции по сборке, запуску и управлению, а также примеры использования и методику тестирования.

Программа полностью удовлетворяет поставленной задаче и может быть использована в качестве удобного инструмента для навигации и управления файловой системой в консольной среде.

СПИСОК ЛИТЕРАТУРНЫХ ИСТОЧНИКОВ

- [1] Информатика учебное пособие [Электронный ресурс]. – Режим доступа: <https://tpt.tom.ru/sved/umk/obscheobraz/uchebnik/osnc.htm>. – Дата доступа: 24.02.2026.
- [2] Tecmint [Электронный ресурс]. – Режим доступа: <https://www.tecmint.com/linux-terminal-file-managers/>. – Дата доступа: 24.02.2026.
- [3] GitHub [Электронный ресурс]. – Режим доступа: <https://github.com/yetone/bubbletea>. – Дата доступа: 24.02.2026.
- [4] University of Hawaii [Электронный ресурс]. – Режим доступа: <https://www2.hawaii.edu/~esb/2004spring.ics412/mar31.html>. – Дата доступа: 24.02.2026.
- [5] Специальная астрофизическая обсерватория [Электронный ресурс]. – Режим доступа: <https://jet.sao.ru/hq/sts/linux/doc/kernel/kernel24/lki-3.html#ss3.3>. – Дата доступа: 24.02.2026.
- [6] ProgrammerSought [Электронный ресурс]. – Режим доступа: <https://www.programmersought.com/article/88486698985/>. – Дата доступа: 24.02.2026.
- [7] ManPages Debian [Электронный ресурс]. – Режим доступа: <https://manpages.debian.org/unstable/ncurses-bin/terminfo.5.en.html>. – Дата доступа: 24.02.2026.
- [8] Manuals OpenSuse [Электронный ресурс]. – Режим доступа: <https://manpages.opensuse.org/Tumbleweed/ncurses-devel/wredrawln.3ncurses.en.html>. – Дата доступа: 24.02.2026.
- [9] Чешский технический университет [Электронный ресурс]. – Режим доступа: https://wiki.control.fel.cvut.cz/pos/os_api/unix-4.html. – Дата доступа: 24.02.2026.
- [10] Таненбаум, Э. Операционные системы: Разработка и реализация. / Таненбаум Э., Уэзеролл Д. – 3-е изд. – СПб.: Питер, 2006. – 1054 с.
- [11] Лав, Р. Ядро Linux: описание процесса разработки. / Лав Р., – 3-е изд. – Диалектика-Вильямс, 2013. – 496 с.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программного кода

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <algorithm>
#include <memory>
#include <set>
#include <functional>
#include <cstring>
#include <cerrno>
#include <locale>
#include <cwchar>
#include <cctype>
#include <sys/types.h>
#include <sys/stat.h>
#include <dirent.h>
#include <unistd.h>
#include <fcntl.h>
#include <libgen.h>
#include <curses.h>
#include <locale>

std::wstring utf8_to_wstring(const std::string& str) {
    if (str.empty()) return L"";
    size_t len = mbstowcs(nullptr, str.c_str(), 0);
    if (len == (size_t)-1) return L"";
    std::vector<wchar_t> buf(len + 1);
    mbstowcs(buf.data(), str.c_str(), len + 1);
    return std::wstring(buf.data());
}

std::string wstring_to_utf8(const std::wstring& wstr) {
    if (wstr.empty()) return "";
    size_t len = wcstombs(nullptr, wstr.c_str(), 0);
```

```

    if (len == (size_t)-1) return "";
    std::vector<char> buf(len + 1);
    wcstombs(buf.data(), wstr.c_str(), len + 1);
    return std::string(buf.data());
}

std::wstring truncateWithEllipsis(const std::wstring& str, int maxlen) {
    if (maxlen <= 0) return L"";
    if ((int)str.length() <= maxlen) return str;
    if (maxlen < 4) return str.substr(0, maxlen);
    return str.substr(0, maxlen - 3) + L"...";
}

std::ofstream logFile;
void log(const std::string& msg) {
    if (logFile.is_open()) {
        logFile << msg << std::endl;
    }
}

struct FileEntry {
    std::wstring name;
    bool isDirectory;
    off_t size;
    mode_t mode;
};

struct TreeNode {
    std::wstring name;
    std::string fullPath;
    bool isDirectory;
    bool expanded;
    std::vector<std::unique_ptr<TreeNode>> children;
    TreeNode* parent;

    TreeNode(const std::string& path, const std::wstring& n, bool isDir,
        TreeNode* p = nullptr)

```

```

        : name(n), fullPath(path), isDirectory(isDir), expanded(false),
parent(p) {}

void loadChildren() {
    if (!isDirectory || expanded) return;
    children.clear();
    DIR* dir = opendir(fullPath.c_str());
    if (!dir) {
        log("Error opening dir " + fullPath + ": " + strerror(errno));
        return;
    }
    struct dirent* entry;
    while ((entry = readdir(dir)) != nullptr) {
        if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name,
"..") == 0)
            continue;
        std::string childPath = fullPath + "/" + entry->d_name;
        struct stat st;
        if (stat(childPath.c_str(), &st) == -1) {
            log("stat error on " + childPath + ": " + strerror(errno));
            continue;
        }
        bool isDir = S_ISDIR(st.st_mode);
        std::wstring wname = utf8_to_wstring(entry->d_name);
        children.push_back(std::make_unique<TreeNode>(childPath, wname,
isDir, this));
    }
    closedir(dir);
    std::sort(children.begin(), children.end(),
        [](const auto& a, const auto& b) {
            if (a->isDirectory != b->isDirectory)
                return a->isDirectory > b->isDirectory;
            return a->name < b->name;
        });
    expanded = true;
}

void unloadChildren() {
    if (!isDirectory) return;

```

```

        children.clear();
        expanded = false;
    }
};

class FileManager {
private:
    std::unique_ptr<TreeNode> root;
    std::vector<TreeNode*> flatList;
    int selectedIndex;
    int scrollOffset;
    bool treeMode;
    std::string currentPath;
    std::vector<FileEntry> flatEntries;
    WINDOW* listWin;
    WINDOW* statusWin;
    int maxY, maxX;
    std::wstring statusMsg;
    std::set<std::string> expandedPaths;

    void buildFlatListFromTree(TreeNode* node, int depth) {
        if (!node) return;
        flatList.push_back(node);
        if (node->isDirectory && node->expanded) {
            for (auto& child : node->children)
                buildFlatListFromTree(child.get(), depth + 1);
        }
    }

    void updateFlatList() {
        flatList.clear();
        if (treeMode && root)
            buildFlatListFromTree(root.get(), 0);
    }

    void saveExpandedState() {
        expandedPaths.clear();
        if (!root) return;

```

```

std::function<void(TreeNode*)> traverse = [&](TreeNode* node) {
    if (node->expanded)
        expandedPaths.insert(node->fullPath);
    for (auto& child : node->children)
        traverse(child.get());
};
traverse(root.get());
}

void restoreExpandedState() {
    if (!root) return;
    std::function<void(TreeNode*)> traverse = [&](TreeNode* node) {
        if (expandedPaths.count(node->fullPath)) {
            node->expanded = true;
            node->loadChildren();
        }
        for (auto& child : node->children)
            traverse(child.get());
    };
    traverse(root.get());
    updateFlatList();
}

void loadTree(const std::string& startPath) {
    char realBuf[PATH_MAX];
    std::string realPath;
    if (realpath(startPath.c_str(), realBuf))
        realPath = realBuf;
    else
        realPath = startPath;

    std::wstring rootName =
utf8_to_wstring(realPath.substr(realPath.find_last_of('/') + 1));
    if (rootName.empty()) rootName = L"/";
    root = std::make_unique<TreeNode>(realPath, rootName, true, nullptr);
    root->loadChildren();
    updateFlatList();
}

```

```

void loadFlatDirectory() {
    flatEntries.clear();

    DIR* dir = opendir(currentPath.c_str());
    if (!dir) {
        statusMsg = L"Error: cannot open directory " +
utf8_to_wstring(currentPath) + L" - " + utf8_to_wstring(strerror(errno));
        log("Error: cannot open directory " + currentPath + " - " +
strerror(errno));
        return;
    }

    struct dirent* entry;
    while ((entry = readdir(dir)) != nullptr) {
        if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name,
"..") == 0)
            continue;

        std::string fullPath = currentPath + "/" + entry->d_name;
        struct stat st;
        if (stat(fullPath.c_str(), &st) == -1) {
            log("stat error on " + fullPath + ": " + strerror(errno));
            continue;
        }

        FileEntry fe;
        fe.name = utf8_to_wstring(entry->d_name);
        fe.isDirectory = S_ISDIR(st.st_mode);
        fe.size = st.st_size;
        fe.mode = st.st_mode;
        flatEntries.push_back(fe);
    }

    closedir(dir);

    std::sort(flatEntries.begin(), flatEntries.end(),
        [](const FileEntry& a, const FileEntry& b) {
            if (a.isDirectory != b.isDirectory)
                return a.isDirectory > b.isDirectory;
            return a.name < b.name;
        });

    if (selectedIndex >= (int)flatEntries.size())
        selectedIndex = flatEntries.empty() ? 0 : flatEntries.size() - 1;
    if (selectedIndex < 0) selectedIndex = 0;
    scrollOffset = 0;
}

```



```

}

void displayList() {
    werase(listWin);
    int visibleRows = maxY - 3;
    if (visibleRows < 1) visibleRows = 1;
    int startIdx = scrollOffset;
    int endIdx;

    if (treeMode) {
        endIdx = std::min((int)flatList.size(), startIdx + visibleRows);
        for (int i = startIdx; i < endIdx; ++i) {
            TreeNode* node = flatList[i];
            int depth = 0;
            TreeNode* p = node->parent;
            while (p) { depth++; p = p->parent; }
            std::wstring indent(depth * 2, L' ');
            std::wstring prefix = node->isDirectory ? (node->expanded ?
L"[-] " : L"[+] ") : L"    ";
            std::wstring line = indent + prefix + node->name;
            int maxLineWidth = maxX - 1;
            if (maxLineWidth < 1) maxLineWidth = 1;
            line = truncateWithEllipsis(line, maxLineWidth);
            int row = i - startIdx;
            if (i == selectedIndex) {
                watttron(listWin, A_REVERSE);
                mvwaddwstr(listWin, row, 0, line.c_str());
                wattroff(listWin, A_REVERSE);
            } else {
                mvwaddwstr(listWin, row, 0, line.c_str());
            }
        }
    } else {
        endIdx = std::min((int)flatEntries.size(), startIdx +
visibleRows);
        for (int i = startIdx; i < endIdx; ++i) {
            const auto& e = flatEntries[i];
            std::wstring prefix = e.isDirectory ? L"[DIR] " : L"[FILE]
";

```

```

        std::wstring line = prefix + e.name;
        int maxLineWidth = maxX - 1;
        if (maxLineWidth < 1) maxLineWidth = 1;
        line = truncateWithEllipsis(line, maxLineWidth);
        int row = i - startIdx;
        if (i == selectedIndex) {
            wattron(listWin, A_REVERSE);
            mvwaddwstr(listWin, row, 0, line.c_str());
            wattroff(listWin, A_REVERSE);
        } else {
            mvwaddwstr(listWin, row, 0, line.c_str());
        }
    }
    wrefresh(listWin);
}

void displayStatus() {
    werase(statusWin);
    std::wstring line0, line1, line2;
    if (treeMode) {
        line0 = L"Mode: TREE";
        if (root)
            line0 += L" Path: " + utf8_to_wstring(root->fullPath);
    } else {
        line0 = L"Mode: FLAT";
        line0 += L" Path: " + utf8_to_wstring(currentPath);
    }
    line1 = L"Status: " + statusMsg;
    line2 = L"Commands: ↑↓ - navigate, Enter - open/expand, Backspace -
parent, c - copy, m - move, d - delete, n - new, t - toggle tree/flat, q -
quit";

    line0 = truncateWithEllipsis(line0, maxX);
    line1 = truncateWithEllipsis(line1, maxX);
    line2 = truncateWithEllipsis(line2, maxX);

    mvwaddwstr(statusWin, 0, 0, line0.c_str());

```

```

        mvwaddwstr(statusWin, 1, 0, line1.c_str());
        mvwaddwstr(statusWin, 2, 0, line2.c_str());
        wrefresh(statusWin);
    }

    void adjustScroll() {
        int visibleRows = maxY - 3;
        if (visibleRows < 1) visibleRows = 1;
        if (selectedIndex < scrollOffset)
            scrollOffset = selectedIndex;
        else if (selectedIndex >= scrollOffset + visibleRows)
            scrollOffset = selectedIndex - visibleRows + 1;
        if (scrollOffset < 0) scrollOffset = 0;
        int totalItems = treeMode ? flatList.size() : flatEntries.size();
        if (scrollOffset > totalItems - visibleRows && totalItems > 0)
            scrollOffset = std::max(0, totalItems - visibleRows);
    }

    std::wstring getSelectedName() {
        if (treeMode && selectedIndex >= 0 && selectedIndex <
(int)flatList.size())
            return flatList[selectedIndex]->name;
        if (!treeMode && selectedIndex >= 0 && selectedIndex <
(int)flatEntries.size())
            return flatEntries[selectedIndex].name;
        return L"";
    }

    std::string getSelectedPath() {
        if (treeMode && selectedIndex >= 0 && selectedIndex <
(int)flatList.size())
            return flatList[selectedIndex]->fullPath;
        if (!treeMode && selectedIndex >= 0 && selectedIndex <
(int)flatEntries.size())
            return currentPath + "/" +
wstring_to_utf8(flatEntries[selectedIndex].name);
        return "";
    }

    std::string getCreationPath() {

```

```

    if (treeMode) {
        if (selectedIndex >= 0 && selectedIndex < (int)flatList.size()) {
            TreeNode* node = flatList[selectedIndex];
            if (node->isDirectory)
                return node->fullPath;
            else {
                std::string parent = node->fullPath;
                size_t pos = parent.find_last_of('/');
                if (pos != std::string::npos)
                    parent = parent.substr(0, pos);
                return parent;
            }
        }
        return root ? root->fullPath : ".";
    } else {
        return currentPath;
    }
}

std::string getDefaultDestinationPath() {
    if (treeMode && selectedIndex >= 0 && selectedIndex <
(int)flatList.size()) {
        TreeNode* node = flatList[selectedIndex];
        if (node->isDirectory)
            return node->fullPath;
        else {
            std::string parent = node->fullPath;
            size_t pos = parent.find_last_of('/');
            if (pos != std::string::npos)
                parent = parent.substr(0, pos);
            return parent;
        }
    }
    return currentPath;
}

bool isSelectedDirectory() {

```

```

        if (treeMode && selectedIndex >= 0 && selectedIndex <
(int)flatList.size())
            return flatList[selectedIndex]->isDirectory;

        if (!treeMode && selectedIndex >= 0 && selectedIndex <
(int)flatEntries.size())
            return flatEntries[selectedIndex].isDirectory;

        return false;
    }

    std::wstring inputStringWithDefault(const std::wstring& prompt, const
std::wstring& defaultValue) {
        echo();
        curs_set(1);
        int height = 3, width = maxX - 4;
        if (width < 10) width = 10;
        int startY = maxY / 2 - 1;
        if (startY < 0) startY = 0;
        int startX = 2;
        WINDOW* inputWin = newwin(height, width, startY, startX);
        box(inputWin, 0, 0);
        mvwaddwstr(inputWin, 1, 1, (prompt + L": ").c_str());
        wrefresh(inputWin);

        std::wstring current = defaultValue;
        int curPos = current.length();
        mvwaddwstr(inputWin, 1, 1 + prompt.length() + 2, current.c_str());
        wmove(inputWin, 1, 1 + prompt.length() + 2 + curPos);
        wrefresh(inputWin);

        int ch;
        while ((ch = wgetch(inputWin)) != '\n' && ch != KEY_ENTER) {
            switch (ch) {
                case KEY_LEFT:
                    if (curPos > 0) curPos--;
                    break;
                case KEY_RIGHT:
                    if (curPos < (int)current.length()) curPos++;
                    break;
                case KEY_BACKSPACE:

```

```

        case 127:
            if (curPos > 0) {
                current.erase(curPos - 1, 1);
                curPos--;
            }
            break;
        case KEY_DC:
            if (curPos < (int)current.length())
                current.erase(curPos, 1);
            break;
        case 21:
            current.clear();
            curPos = 0;
            break;
        default:
            if (ch >= 32 && ch <= 126) {
                current.insert(curPos, 1, (wchar_t)ch);
                curPos++;
            }
            break;
    }

    mvwaddwstr(inputWin, 1, 1 + prompt.length() + 2,
std::wstring(width - 2 - prompt.length() - 2, L' ').c_str());
    mvwaddwstr(inputWin, 1, 1 + prompt.length() + 2,
current.c_str());

    wmove(inputWin, 1, 1 + prompt.length() + 2 + curPos);
    wrefresh(inputWin);
}

delwin(inputWin);
curs_set(0);
noecho();
return current;
}

bool exists(const std::string& path) {
    struct stat st;
    return stat(path.c_str(), &st) == 0;
}

```

```

    }

    bool copyFile(const std::string& src, const std::string& dst) {
        int fdSrc = open(src.c_str(), O_RDONLY);
        if (fdSrc == -1) {
            statusMsg = L"Error: cannot open source file " +
                utf8_to_wstring(src) + L" - " + utf8_to_wstring(strerror(errno));
            log("Error: cannot open source file " + src + " - " +
                strerror(errno));
            return false;
        }
        int fdDst = open(dst.c_str(), O_WRONLY | O_CREAT | O_TRUNC, 0644);
        if (fdDst == -1) {
            statusMsg = L"Error: cannot create destination file " +
                utf8_to_wstring(dst) + L" - " + utf8_to_wstring(strerror(errno));
            log("Error: cannot create destination file " + dst + " - " +
                strerror(errno));
            close(fdSrc);
            return false;
        }
        char buffer[8192];
        ssize_t n;
        while ((n = read(fdSrc, buffer, sizeof(buffer))) > 0) {
            if (write(fdDst, buffer, n) != n) {
                statusMsg = L"Error: write error to " + utf8_to_wstring(dst)
                    + L" - " + utf8_to_wstring(strerror(errno));
                log("Error: write error to " + dst + " - " +
                    strerror(errno));
                close(fdSrc);
                close(fdDst);
                return false;
            }
        }
        if (n == -1) {
            statusMsg = L"Error: read error from " + utf8_to_wstring(src) +
                L" - " + utf8_to_wstring(strerror(errno));
            log("Error: read error from " + src + " - " + strerror(errno));
        }
        close(fdSrc);
        close(fdDst);
        if (n != -1) {

```

```

        log("Copied file: " + src + " -> " + dst);
    }
    return n != -1;
}

bool copyDirectory(const std::string& src, const std::string& dst) {
    DIR* dir = opendir(src.c_str());
    if (!dir) {
        statusMsg = L"Error: cannot open source directory " +
            utf8_to_wstring(src) + L" - " + utf8_to_wstring(strerror(errno));
        log("Error: cannot open source directory " + src + " - " +
            strerror(errno));
        return false;
    }
    if (mkdir(dst.c_str(), 0755) == -1 && errno != EEXIST) {
        statusMsg = L"Error: cannot create destination directory " +
            utf8_to_wstring(dst) + L" - " + utf8_to_wstring(strerror(errno));
        log("Error: cannot create destination directory " + dst + " - " +
            strerror(errno));
        closedir(dir);
        return false;
    }
    struct dirent* entry;
    bool success = true;
    while ((entry = readdir(dir)) != nullptr) {
        if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name,
"..") == 0)
            continue;
        std::string srcPath = src + "/" + entry->d_name;
        std::string dstPath = dst + "/" + entry->d_name;
        struct stat st;
        if (stat(srcPath.c_str(), &st) == -1) {
            log("stat error on " + srcPath + ": " + strerror(errno));
            success = false;
            continue;
        }
        if (S_ISDIR(st.st_mode)) {
            if (!copyDirectory(srcPath, dstPath))
                success = false;
        } else {

```



```

        if (!copyFile(srcPath, dstPath))
            success = false;
    }
}

closedir(dir);
if (success) {
    log("Copied directory: " + src + " -> " + dst);
}
return success;
}

bool removeDirectory(const std::string& path) {
    DIR* dir = opendir(path.c_str());
    if (!dir) {
        statusMsg = L"Error: cannot open directory " +
utf8_to_wstring(path) + L" - " + utf8_to_wstring(strerror(errno));
        log("Error: cannot open directory " + path + " - " +
strerror(errno));
        return false;
    }
    struct dirent* entry;
    bool success = true;
    while ((entry = readdir(dir)) != nullptr) {
        if (strcmp(entry->d_name, ".") == 0 || strcmp(entry->d_name,
"..") == 0)
            continue;
        std::string fullPath = path + "/" + entry->d_name;
        struct stat st;
        if (stat(fullPath.c_str(), &st) == -1) {
            log("stat error on " + fullPath + ": " + strerror(errno));
            success = false;
            continue;
        }
        if (S_ISDIR(st.st_mode)) {
            if (!removeDirectory(fullPath))
                success = false;
        } else {
            if (unlink(fullPath.c_str()) == -1) {

```

```

        log("unlink error on " + fullPath + ": " +
strerror(errno));
        success = false;
    }
}
}
closedir(dir);
if (rmdir(path.c_str()) == -1) {
    log("rmdir error on " + path + ": " + strerror(errno));
    success = false;
}
if (success) {
    log("Removed directory: " + path);
}
return success;
}

```

```

void refreshAfterResize() {
    getmaxyx(stdscr, maxY, maxX);
    if (maxY < 4) maxY = 4;
    if (maxX < 10) maxX = 10;
    if (listWin) delwin(listWin);
    if (statusWin) delwin(statusWin);
    listWin = newwin(maxY-3, maxX, 0, 0);
    statusWin = newwin(3, maxX, maxY-3, 0);
    scrollok(listWin, FALSE);
    adjustScroll();
    displayList();
    displayStatus();
}

```

```

void updateTreeAfterOperation() {
    if (!treeMode) return;
    saveExpandedState();
    loadTree(currentPath);
    restoreExpandedState();
}

```

```

public:

    FileManager(const std::string& startPath) : selectedIndex(0),
        scrollOffset(0), treeMode(true) {

        char realBuf[PATH_MAX];
        if (realpath(startPath.c_str(), realBuf))
            currentPath = realBuf;
        else
            currentPath = startPath;

        loadTree(currentPath);
        loadFlatDirectory();
    }

    void run() {
        initscr();
        cbreak();
        noecho();
        keypad(stdscr, TRUE);
        curs_set(0);
        getmaxyx(stdscr, maxY, maxX);
        if (maxY < 4) maxY = 4;
        if (maxX < 10) maxX = 10;
        clear();
        refresh();

        listWin = newwin(maxY-3, maxX, 0, 0);
        statusWin = newwin(3, maxX, maxY-3, 0);
        scrollok(listWin, FALSE);
        adjustScroll();
        displayList();
        displayStatus();

        int ch;
        while ((ch = getch()) != 'q') {
            if (ch == KEY_RESIZE) {
                refreshAfterResize();
                continue;
            }
        }
    }

```

```

int cmd = ch;

if (ch >= 'A' && ch <= 'Z') cmd = tolower(ch);

switch (cmd) {
    case KEY_UP:
        if (selectedIndex > 0) { --selectedIndex;
adjustScroll(); }
        break;
    case KEY_DOWN:
        if (treeMode && selectedIndex < (int)flatList.size()-1) {
            ++selectedIndex; adjustScroll();
        } else if (!treeMode && selectedIndex <
(int)flatEntries.size()-1) {
            ++selectedIndex; adjustScroll();
        }
        break;
    case '\n':
    case KEY_ENTER:
        if (treeMode) {
            if (selectedIndex >= 0 && selectedIndex <
(int)flatList.size()) {
                TreeNode* node = flatList[selectedIndex];
                if (node->isDirectory) {
                    if (node->expanded)
                        node->unloadChildren();
                    else
                        node->loadChildren();
                }
                updateFlatList();
                for (size_t i = 0; i < flatList.size(); ++i)
                {
                    if (flatList[i] == node) {
                        selectedIndex = i;
                        break;
                    }
                }
                adjustScroll();
            } else {
                statusMsg = L"Selected file: " + node->name;
            }
        }
    }
}

```

```

        }
    } else {
        if (selectedIndex >= 0 && selectedIndex <
(int)flatEntries.size() &&
            flatEntries[selectedIndex].isDirectory) {
            std::string newPath = currentPath + "/" +
wstring_to_utf8(flatEntries[selectedIndex].name);
            currentPath = newPath;
            selectedIndex = 0;
            loadFlatDirectory();
        } else if (selectedIndex >= 0 && selectedIndex <
(int)flatEntries.size() &&
            !flatEntries[selectedIndex].isDirectory) {
            statusMsg = L"Selected file: " +
flatEntries[selectedIndex].name;
        }
    }
    break;
case KEY_BACKSPACE:
case 127:
    if (treeMode) {
        if (root && root->fullPath != "/") {
            std::string parent = currentPath;
            size_t pos = parent.find_last_of('/');
            if (pos != std::string::npos && pos > 0)
                parent = parent.substr(0, pos);
            else
                parent = "/";
            currentPath = parent;
            loadTree(currentPath);
            loadFlatDirectory();
            selectedIndex = 0;
            adjustScroll();
        } else {
            statusMsg = L"Already at root";
        }
    } else {
        std::string parent = currentPath;
        size_t pos = parent.find_last_of('/');

```

```

        if (pos != std::string::npos && pos > 0)
            parent = parent.substr(0, pos);
        else
            parent = "/";
        if (parent != currentPath) {
            currentPath = parent;
            selectedIndex = 0;
            loadFlatDirectory();
        } else {
            statusMsg = L"Already at root";
        }
    }
    break;
case 't':
    treeMode = !treeMode;
    selectedIndex = 0;
    scrollOffset = 0;
    if (treeMode) updateFlatList();
    else loadFlatDirectory();
    break;
case 'c': {
    std::string src = getSelectedPath();
    if (src.empty()) {
        statusMsg = L"No file selected";
        break;
    }
    std::string defaultDst = getDefaultDestinationPath();
    std::wstring dstW = inputStringWithDefault(L"Enter
destination path", utf8_to_wstring(defaultDst));
    if (dstW.empty()) {
        statusMsg = L"Copy cancelled";
        break;
    }
    std::string dst = wstring_to_utf8(dstW);
    if (exists(dst)) {
        statusMsg = L"Error: destination already exists";
    } else {

```

```

        bool ok = isSelectedDirectory() ? copyDirectory(src,
dst) : copyFile(src, dst);

        if (ok) {
            statusMsg = L"Copied successfully";
            if (treeMode) {
                updateTreeAfterOperation();
            } else {
                loadFlatDirectory();
            }
        }
    }
    break;
}

case 'm': {
    std::string src = getSelectedPath();
    if (src.empty()) {
        statusMsg = L"No file selected";
        break;
    }

    std::wstring defaultName = getSelectedName();
    std::wstring dstW = inputStringWithDefault(L"Enter new
name/path", defaultName);
    if (dstW.empty()) {
        statusMsg = L"Move cancelled";
        break;
    }

    std::string dst = wstring_to_utf8(dstW);
    if (rename(src.c_str(), dst.c_str()) == -1) {
        statusMsg = L"Error moving: " +
utf8_to_wstring(strerror(errno));
        log("Error moving: " + std::string(strerror(errno)));
    } else {
        statusMsg = L"Moved successfully";
        log("Moved " + src + " -> " + dst);
        if (treeMode) {
            updateTreeAfterOperation();
        } else {
            loadFlatDirectory();
        }
    }
}

```

```

        }
        break;
    }
    case 'd': {
        std::string target = getSelectedPath();
        if (target.empty()) {
            statusMsg = L"No file selected";
            break;
        }
        std::string confirm =
wstring_to_utf8(inputStringWithDefault(L"Confirm delete (y/n)", L""));
        if (confirm == "y" || confirm == "Y") {
            bool ok = isSelectedDirectory() ?
removeDirectory(target) : (unlink(target.c_str()) == 0);
            if (ok) {
                statusMsg = L"Deleted successfully";
                log("Deleted " + target);
                if (treeMode) {
                    updateTreeAfterOperation();
                } else {
                    loadFlatDirectory();
                }
            } else {
                statusMsg = L"Error deleting: " +
utf8_to_wstring(strerror(errno));
                log("Error deleting: " +
std::string(strerror(errno)));
            }
        } else {
            statusMsg = L"Delete cancelled";
        }
        break;
    }
    case 'n': {
        std::string type =
wstring_to_utf8(inputStringWithDefault(L"Enter type (f=file, d=dir)", L""));
        std::string creationPath = getCreationPath();
        if (type == "f") {
            std::string name =
wstring_to_utf8(inputStringWithDefault(L"Enter file name", L""));

```



```

        if (!name.empty()) {
            std::string fullPath = creationPath + "/" + name;
            int fd = open(fullPath.c_str(), O_WRONLY |
O_CREAT | O_EXCL, 0644);
            if (fd == -1) {
                statusMsg = L"Error creating file: " +
utf8_to_wstring(strerror(errno));
                log("Error creating file: " +
std::string(strerror(errno)));
            } else {
                close(fd);
                statusMsg = L"File created";
                log("Created file " + fullPath);
                if (treeMode) {
                    updateTreeAfterOperation();
                } else {
                    loadFlatDirectory();
                }
            }
        }
    } else if (type == "d") {
        std::string name =
wstring_to_utf8(inputStringWithDefault(L"Enter directory name", L""));
        if (!name.empty()) {
            std::string fullPath = creationPath + "/" + name;
            if (mkdir(fullPath.c_str(), 0755) == -1) {
                statusMsg = L"Error creating directory: " +
utf8_to_wstring(strerror(errno));
                log("Error creating directory: " +
std::string(strerror(errno)));
            } else {
                statusMsg = L"Directory created";
                log("Created directory " + fullPath);
                if (treeMode) {
                    updateTreeAfterOperation();
                } else {
                    loadFlatDirectory();
                }
            }
        }
    }
}

```

```

        } else {
            statusMsg = L"Invalid type";
        }
        break;
    }
    default:
        break;
    }
    displayList();
    displayStatus();
}

endwin();
}
};

int main(int argc, char* argv[]) {
    setlocale(LC_ALL, "");
    std::locale::global(std::locale(""));

    logFile.open("file_manager.log", std::ios::app);
    if (!logFile.is_open())
        std::cerr << "Warning: Could not open log file" << std::endl;

    std::string startPath = ".";
    if (argc > 1)
        startPath = argv[1];

    FileManager fm(startPath);
    fm.run();

    if (logFile.is_open()) logFile.close();
    return 0;
}

```

ПРИЛОЖЕНИЕ Б
(обязательное)
Функциональная схема навигатора

ПРИЛОЖЕНИЕ В
(обязательное)
Блок-схема алгоритма обработки событий ввода и
перерисовки экрана

ПРИЛОЖЕНИЕ Г
(обязательное)
Графический интерфейс пользователя

ПРИЛОЖЕНИЕ Д
(обязательное)
Ведомость курсового проекта